PCS Loan Module — Developer Handover

V4.65 · 12 October 2026

A private-credit loan accounting sub-ledger. It takes loan terms and cashflows — built in the module or imported from a servicing system — applies IFRS 9 (and US GAAP CECL, Canadian ASPE, AASB), and produces balanced journal entries for FIS Investran.

---

1 · Read this first

Three things a developer needs to know before touching anything.

The application is eight files. There is no build step, no bundler, no framework. The browser loads nine files from our own origin and nothing else. If you are looking for a `src/` directory and a `package.json` for the front end, there isn't one — that is a deliberate property of the thing, not an omission.

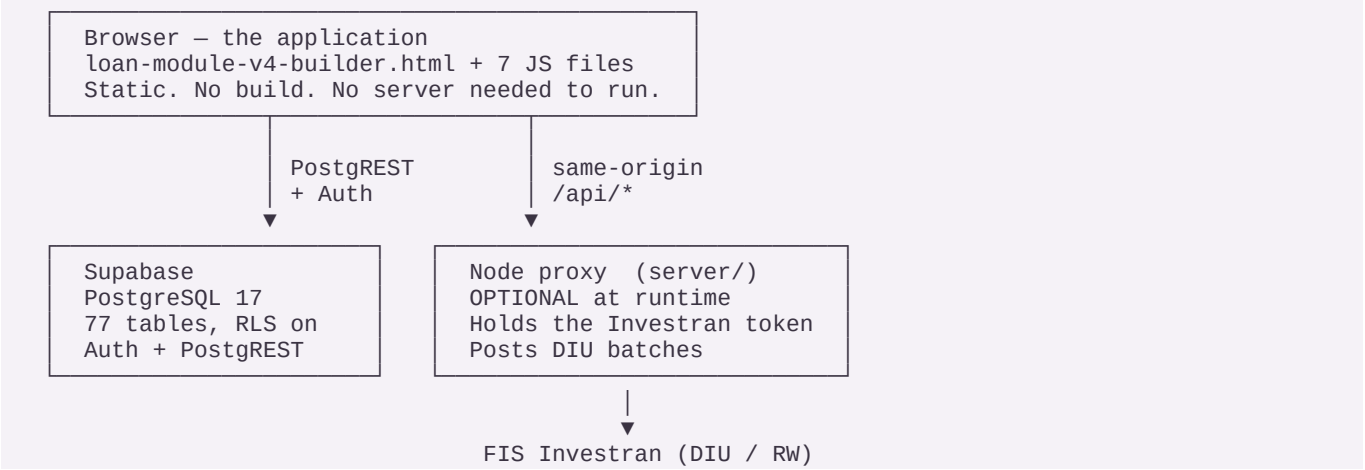
There are credentials in the working repo that must be rotated. See §9. Do this before anything else.

The repo you were handed is not the repo you should keep. The working tree contains committed `node_modules`, API probe scratch output, sample PDFs, two superseded builder versions and a stale publish script. `package-for-handover.sh` extracts only what runs. §10 lists what to delete.

---

2 · Architecture

Three layers, each independently deployable.



The browser layer is the whole application: UI, the accounting engine, the Excel parser, the GL mapper, the offline assistant. It talks to Supabase directly over PostgREST using a publishable anon key, with row-level security doing the access control.

Supabase is the system of record for deals, runs, journals and configuration. It is a standard Postgres — nothing Supabase-specific is required beyond PostgREST-compatible REST and Auth. The app has adapter stubs for raw PostgREST, Hasura and a custom REST wrapper (gear icon → database settings), though only the Supabase path is exercised.

The Node proxy is not needed to run the application. It is needed to (a) post journal batches to Investran DIU and (b) use the AI assistant in online mode. Without it the module still calculates, journalises and stores everything; it just cannot push to the ledger, and the assistant falls back to its offline knowledge base.

---

3 · What to deploy

The application — nine files, static hosting

File	Role

`loan-module-v4-builder.html`	**Entry point.** The entire UI and most application logic. Large.
`loan-module-engine.js`	The accounting engine: schedules, EIR, ECL, modification, journals
`loan-module-instruments.js`	Instrument definitions and demo presets
`loan-module-analytics.js`	Portfolio and dashboard calculations
`portf-excel-parser.js`	The external-deal workbook reader (v0.3–v0.5 templates)
`demo-assistant-kb.js`	Assistant knowledge base — workflow Q&A
`demo-assistant-kb-ifrs.js`	Assistant knowledge base — 256 IFRS Q&A, generated
`demo-assistant-system-prompt.js`	Assistant prompt for online mode
`build-manifest.json`	Per-file hashes so the running page can report a stale deploy

There is no `index.html`. The entry point is `loan-module-v4-builder.html`. Either rename it, add an index that redirects, or configure the host's default document. Decide this early — it affects every link you hand out.

Any static host works: S3 + CloudFront, nginx, IIS, GitHub Pages, a shared drive. The only requirement is that all nine files sit in the same directory and are served over the same origin.

The proxy — optional, Node 18+

`server/server.js`, an Express app. Deploy only if you need DIU posting or the online assistant.

```
cd server
npm ci                  # not npm install — the lockfile is authoritative
cp .env.example .env    # then fill it in, see §4
npm start               # listens on 4318 by default
```

It must be reachable from the browser at the **same origin** as the app under `/api/`, because the app calls `/api/anthropic` and `/api/diu/*` as relative paths. In production that means a reverse proxy rule, not CORS.

The database

See §5. You need a PostgreSQL with the schema, PostgREST in front of it, and an auth provider issuing JWTs.

---

4 · Configuration

Browser

The Supabase URL and anon key are **compiled into the HTML** as defaults:

```
const DEFAULT_URL  = 'https://prljhhyfgzipjnfpztfu.supabase.co';
const DEFAULT_ANON = 'eyJhbGciOi…';          // publishable anon key
```

Users override these at runtime via the gear icon; the override lives in `localStorage` under `pcs_v3_db_config`.

The anon key being public is by design — that is what a Supabase anon key is. The access control is row-level security, not key secrecy. This is only safe while RLS is enabled on every table; see §5.

For your own environment, change `DEFAULT_URL` and `DEFAULT_ANON` in the HTML, or ship with them blank and make users configure. **Never put a service-role key here.** The app's own settings panel says so and it is the single most important rule on this layer.

Server — `server/.env`

Variable	Required	Purpose
`INVESTRAN_BASE_URL`	yes, for DIU	Tenant base, e.g. `https://…/api`
`INVESTRAN_BEARER_TOKEN`	one of	Static bearer token
`INVESTRAN_CLIENT_ID` / `INVESTRAN_CLIENT_SECRET`	one of	OAuth client-credentials, preferred

`ANTHROPIC_API_KEY`	no	Online assistant. Omit and it runs offline
`PORT`	no	Defaults to 4318

`.env` is in `server/.gitignore` and must stay there.

---

5 · The database

PostgreSQL 17 · 77 tables · 227 RLS policies · 57 enum types · 101 migrations.

Row-level security is enabled on **all 77 tables**. Three tables were missing it as of 12 October and were corrected — if you fork from an earlier snapshot, check `pg_tables.rowsecurity` before exposing the anon key anywhere.

Access control model:

- `is_admin()` — admin override
- `in_team(team_id)` — team-scoped workspace configuration
- `owns_deal(deal_id)` — per-deal access via `deal_user_assignments`

Some early policies grant `{anon, authenticated}` with `using (true)` — a hangover from the demo phase. **Audit these before any non-demo use.** `rfr_fixings` is one example.

Getting a database

The authoritative route is the Supabase CLI against the existing project:

```
supabase link --project-ref prljhhyfgzipjnfpztfu
supabase db dump --schema public       > schema.sql     # structure
supabase db dump --schema public --data-only > seed.sql # reference data
```

Use the CLI rather than a hand-assembled script. It captures functions, triggers, grants, sequences and extensions — the things a reconstructed `CREATE TABLE` dump silently omits, producing a file that looks complete and does not rebuild.

`SCHEMA-REFERENCE.md` accompanies this document: all 57 enums, every table with column and policy counts and current row counts, and the full migration history. It is a **reference for understanding the model**, not a rebuild script.

What is reference data and what is demo data

Reference data to keep: `accounting_frameworks`, `gl_accounts`, `gl_business_events`, `gl_event_mappings`, `gl_frameworks`, `gl_transtype_mappings`, `cashflow_column_config`, `preset_catalog`.

Demo data to drop for a clean environment: `deals` and everything hanging off it, `accounting_runs`, `journal_entries` (18,477 rows), `cashflow_schedules` (222,072 rows), `external_*`, `loan_notices` (2,921 rows).

---

6 · Build and release

There is no compile step, but there **is** a release discipline, and it exists because of two production incidents.

The problem it solves

The app is nine files. Deploying some of them leaves the page looking current — correct version label, no errors — while running older logic beside it. That has happened twice: the HTML shipped, the engine did not, and two accounting fixes were silently absent. Separately, cache-busters sat at `?v=4.4` while the version read 4.8, so browsers kept running cached code and every hash check passed.

The mechanism

`.githooks/pre-commit` — installed per clone, hooks are not cloned:

```
git config core.hooksPath .githooks
```

On each commit it bumps `PCS_VERSION_MINOR`, rewrites every `?v=` cache-buster to match, and writes `build-manifest.json` with a hash per deployable file.

`check-manifest.sh` — run in CI. Fails if the manifest does not describe the tree, which is what happens when someone commits from a clone where the hook was never installed.

`deploy.sh` — copies all nine files, aborts if any is missing, prints hashes to compare against what the browser reports via `await pcsVersion()`.

If you change this

Keep the invariant: **the version shown on screen must be derivable from what the browser actually loaded.** Any release process satisfying that is fine. The current one is crude but it fails loudly, which is the property that matters.

---

7 · The offline assistant

`demo-assistant-kb-ifrs.js` is **generated** from `PCS-IFRS-Demo-Answers.md` by `build-ifrs-kb.py`. Do not hand-edit it. Edit the markdown and regenerate, so the document people read and the answers the assistant gives cannot drift apart.

It appends to `window.FAQ_KB`, so it must load **after** `demo-assistant-kb.js`. Loading it first would silently drop all 256 entries with no error.

---

8 · External deal intake

Deals arrive two ways and must resolve to the same record.

Excel — the v0.5 workbook, seven sheets, parsed in the browser by `portf-excel-parser.js`. Ceiling about 500,000 rows.

API — `GET /deals?changedSince=` and `GET /deals/{id}`, full snapshot per deal, no row limit. Specified in `docs/PortF-feed-contract-v0.3.md`. **Not yet implemented** — the contract is agreed, the consumer is not built. This is the largest single piece of outstanding work.

Imported deals are `source = external` and read-only in the UI. The ledger stops where the feed stops.

---

9 · Security — do these first

1 · Rotate the Investran credentials. `server/.env` in the working repo contains a live bearer token for a UAT tenant. It is gitignored, but it has existed in a working directory on at least one laptop. Rotate it.

2 · Rotate the Anthropic API key in the same file, for the same reason.

3 · Move to OAuth client-credentials for Investran rather than a static bearer token. `server.js` supports both; the quickstart documents the client-credentials path. Use a dedicated service account with the narrowest scope that works — `dataimport.write` for posting.

4 · Audit the permissive RLS policies described in §5 before any real data.

5 · Confirm `.env` was never committed. The ignore rule exists now; check the history:

```
git log --all --full-history -- Incomemodulev3/server/.env
```

If anything comes back, the token is in the history and rotation is not optional.

---

10 · Repo hygiene — delete before adopting

What	Why
`Incomemodulev3/server/node_modules/`	Committed dependencies. `npm ci` from the lockfile instead.
`server/rw-result-*.json`, `diu-validation-*.json`, `meta-*.json`, `swagger-*.json`	API probe output. Scratch, not source.

`server/probe-*.js`, `test-diu.js`, `dump-transaction-meta.js`, `fetch-swagger.js`, `set-token.js`	Exploration scripts from the integration work. Keep if useful, but they are not the product.
`loan-module-v3-builder.html`, `loan-module-v2-builder.html`	Superseded. V4 is the product.
`PUBLISH.sh`	Stale — pushes V3 files to a GitHub repo. Dead.
Sample PDFs, demo workbooks, `.DS_Store`	Demo material. Move to a separate repo if kept.
`Incomemodulev3.zip`	A zip of the project, inside the project.

Note the doubled path: the repo root contains a folder named `Incomemodulev3`, so everything lives at `Incomemodulev3/Incomemodulev3/`. Flatten it.

---

11 · Known defects

Carried forward honestly. None blocks the demo; several would block production.

Accounting correctness

#	Issue
71	**Purchased assets repay at face.** A POCI loan carried at accreted consideration (~76.9m) posts a repayment of the full 100m face. Acquisition was fixed in V4.65; the disposal leg was not.
54	**Repaid loans leave a residual.** Carrying value and allowance do not always reach nil at maturity.
53	**Two modifications fail the 10% test but are booked non-substantial.** The test is measured and reported; the classification does not follow it.
15	**External-deal EIR is not solved against received cashflows.**

Posting and presentation

#	Issue
72	POCI deals report `eclStage: 1`. POCI has no stages — a display default that reads as a measurement.
62	Fee rows bypass the GL mapping configuration in some paths.
60	All-in forecast curves: margin suppression needs review.
21	Export XLSX on an external deal can emit another deal's cashflows.
19	Facility Setup is not imported from the workbook — two silent failures.

Smaller

#1 (unhelpful SOFR timeout message), #16, #18, #39.

The pattern worth knowing

Most defects found in this codebase share one shape: **a value that is read, stored, displayed, and never consulted.** A fee frequency parsed and ignored. A GL mapping screen nothing read. Rate conventions discarded and replaced by zero. A `?v=` tag that stopped matching its version. When something behaves oddly, check whether the value reaching the calculation is the one on screen — that has been the answer more often than any other.

---

12 · Where to start

1. Rotate credentials (§9).

2. `./package-for-handover.sh ~/handover` and read what comes out.

3. Serve `app/` on any static host; open `loan-module-v4-builder.html`.

4. Point it at your own database via the gear icon, or edit the defaults.

5. Load a demo deal, run accounting, inspect the journals. That exercises the whole path.

6. Read `docs/PortF-feed-contract-v0.3.md` — the external feed is the main unbuilt piece.

The accounting logic lives in `loan-module-engine.js`. It is long and densely commented, and the comments explain *why* rather than *what* — several record a specific defect and the reasoning that fixed it. They are the fastest way into the domain.